Національний технічний університет України

«Київський політехнічний інститут
імені Ігоря Сікорського»

Технології розробки на платформі Java

Лабораторна робота 1

Сучасні можливості Java та підхід
Data Oriented Programming

Система управління замовленнями

Виконав
студент групи ІО-41
Гасюк Д.О.
Номер залікової книжки 4104

Київ 2026

Мета роботи:

Навчитися моделювати незмінні дані за допомогою records і sealed types, відокремлювати дані від алгоритмів та реалізовувати вичерпну обробку станів через switch expression із pattern matching. Сформувати текстові результати з використанням text blocks і перевірити програму модульними тестами JUnit 5.

Визначення варіанта:

Номер залікової книжки N = 4104. За умовою номер варіанта визначається остачею від ділення N на 3. Оскільки $4104 = 3 \times 1368 + 0$, отримано V = 0.

Для варіанта 0 реалізовано систему управління замовленнями з чотирма станами: очікування обробки, оплата, відправлення та скасування.

Постановка завдання:

У пакеті ua.kpi.comsys.dop необхідно створити публічний sealed interface OrderStatus та публічні записи Pending, Paid, Shipped і Cancelled. Конструктори перевіряють коректність даних. Окремий клас OrderProcessor перетворює кожен стан на рядок установленого формату.

Метод process має відхиляти null винятком IllegalArgumentException із повідомленням Status cannot be null. Усі дозволені стани обробляються у switch expression без гілки default. Для кожної гілки використовується text block.

Програмні засоби:

Мова реалізації Java, цільова версія 21. Для перевірки використано JDK 25.0.1 із параметром компіляції --release 21 та JDK 21.0.12. Тести побудовано на JUnit Jupiter 5.11.4. Проєкт відкривається в IntelliJ IDEA як Maven-проєкт. Програма запускається через метод Main.main, а тести через вбудовану підтримку JUnit.

Модель даних і принципи DOP:

Підхід Data Oriented Programming розділяє структури даних і код їх обробки. Записи містять лише стан замовлення та перевірки допустимості значень, а форматування зосереджено у класі OrderProcessor.

Sealed interface задає закриту множину реалізацій. Компілятор перевіряє, що switch охоплює кожен дозволений стан. Records автоматично створюють конструктор, методи доступу, equals, hashCode і toString. Компоненти цих записів мають типи String, BigDecimal і LocalDate, які є незмінними; тому стан створених об’єктів неможливо змінити.

Стан	Дані	Умови коректності
Pending	Без компонентів	Додаткові перевірки не потрібні
Paid	paymentId amount	Ідентифікатор не null і не blank; сума не null та більша за нуль
Shipped	trackingCode dispatchDate	Код не null і не blank; дата не null
Cancelled	reason	Причина не null і не blank

Алгоритм обробки:

Спочатку конструктор record перевіряє вхідні значення. Некоректний об’єкт не створюється: одразу виникає IllegalArgumentException. Після цього process перевіряє посилання status і вибирає гілку за типом запису. Record pattern надає доступ до компонентів. Text block задає шаблон, а formatted підставляє значення.

Для суми застосовано BigDecimal.compareTo, оскільки порівняння має залежати від числового значення. Метод toPlainString зберігає десятковий запис без експоненти й округлення. Дата LocalDate відображається у форматі рік-місяць-день.

Структура проєкту

```
lab1-order-system/
  src/main/java/ua/kpi/comsys/dop/
    OrderStatus.java    Pending.java
    Paid.java           Shipped.java
    Cancelled.java      OrderProcessor.java
    Main.java
  src/test/java/ua/kpi/comsys/dop/
    OrderProcessorTest.java
    OrderStatusTest.java
  pom.xml
  README.md
```

Лістинг моделі даних

OrderStatus.java

```
package ua.kpi.comsys.dop;

/** Закрита ієрархія: нові стани потребуватимуть оновлення switch. */
public sealed interface OrderStatus permits Pending, Paid, Shipped, Cancelled {
}
```

Pending.java

```
package ua.kpi.comsys.dop;

/** Початковий стан не містить додаткових даних. */
public record Pending() implements OrderStatus {
}
```

Paid.java

```
package ua.kpi.comsys.dop;

import java.math.BigDecimal;

/** Незмінні дані про виконану оплату. */
public record Paid(String paymentId, BigDecimal amount) implements OrderStatus {
    // Компактний конструктор перевіряє дані перед створенням record.
    public Paid {
        if (paymentId == null || paymentId.isBlank()) {
            throw new IllegalArgumentException("Payment ID cannot be null or blank");
        }
        if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Amount must be greater than zero");
        }
    }
}
```

Лістинг станів відправлення та скасування

Shipped.java

```
package ua.kpi.comsys.dop;

import java.time.LocalDate;

/** Дані про відправлення замовлення. */
public record Shipped(String trackingCode, LocalDate dispatchDate)
        implements OrderStatus {
    public Shipped {
        if (trackingCode == null || trackingCode.isBlank()) {
            throw new IllegalArgumentException("Tracking code cannot be null or blank");
        }
        if (dispatchDate == null) {
            throw new IllegalArgumentException("Dispatch date cannot be null");
        }
    }
}
```

Cancelled.java

```
package ua.kpi.comsys.dop;

/** Причина скасування є обов'язковою. */
public record Cancelled(String reason) implements OrderStatus {
    public Cancelled {
        if (reason == null || reason.isBlank()) {
            throw new IllegalArgumentException("Reason cannot be null or blank");
        }
    }
}
```

Властивості валідації:

Перевірка null виконується перед isBlank і compareTo, тому помилкові дані дають передбачений IllegalArgumentException. Порожні рядки, пробіли, табуляція та переведення рядка не є допустимими ідентифікаторами або причинами скасування.

Непорожні текстові значення зберігаються без прихованого обрізання пробілів. Для дати відправлення умова вимагає лише відсутності null. Обмеження на минулі або майбутні дати не вводяться.

Лістинг обробника станів:

OrderProcessor.java

```
package ua.kpi.comsys.dop;

import java.math.BigDecimal;
import java.time.LocalDate;

/** Формує текстовий опис будь-якого дозволеного стану. */
public class OrderProcessor {
    public String process(OrderStatus status) {
        if (status == null) {
            throw new IllegalArgumentException("Status cannot be null");
        }

        // Record patterns розкладають стан на компоненти; switch є вичерпним.
        return switch (status) {
            case Pending() -> """
                    Order Status: PENDING
                    Details: Order is currently pending processing.
                    """;
            case Paid(String paymentId, BigDecimal amount) -> """
                    Order Status: PAID
                    Details: Payment ID: %s, Amount: $%s
                    """.formatted(paymentId, amount.toPlainString());
            case Shipped(String trackingCode, LocalDate dispatchDate) -> """
                    Order Status: SHIPPED
                    Details: Tracking Code: %s, Dispatch Date: %s
                    """.formatted(trackingCode, dispatchDate);
            case Cancelled(String reason) -> """
                    Order Status: CANCELLED
                    Details: Reason: %s
                    """.formatted(reason);
        };
    }
}
```

Кожна гілка повертає два рядки та завершальне переведення рядка. Шаблони відповідають специфікації, включно з регістром назв станів, пробілами, комами та знаком долара.

Демонстраційна програма:

Main.java

```
package ua.kpi.comsys.dop;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

/** Демонстрація варіанта 0: 4104 % 3 = 0. */
public class Main {
    public static void main(String[] args) {
        OrderProcessor processor = new OrderProcessor();

        System.out.println("Лабораторна робота 1 | Гасюк Д.О. | "
                + "ІО-41 | 4104 | Варіант 0");
        System.out.println();

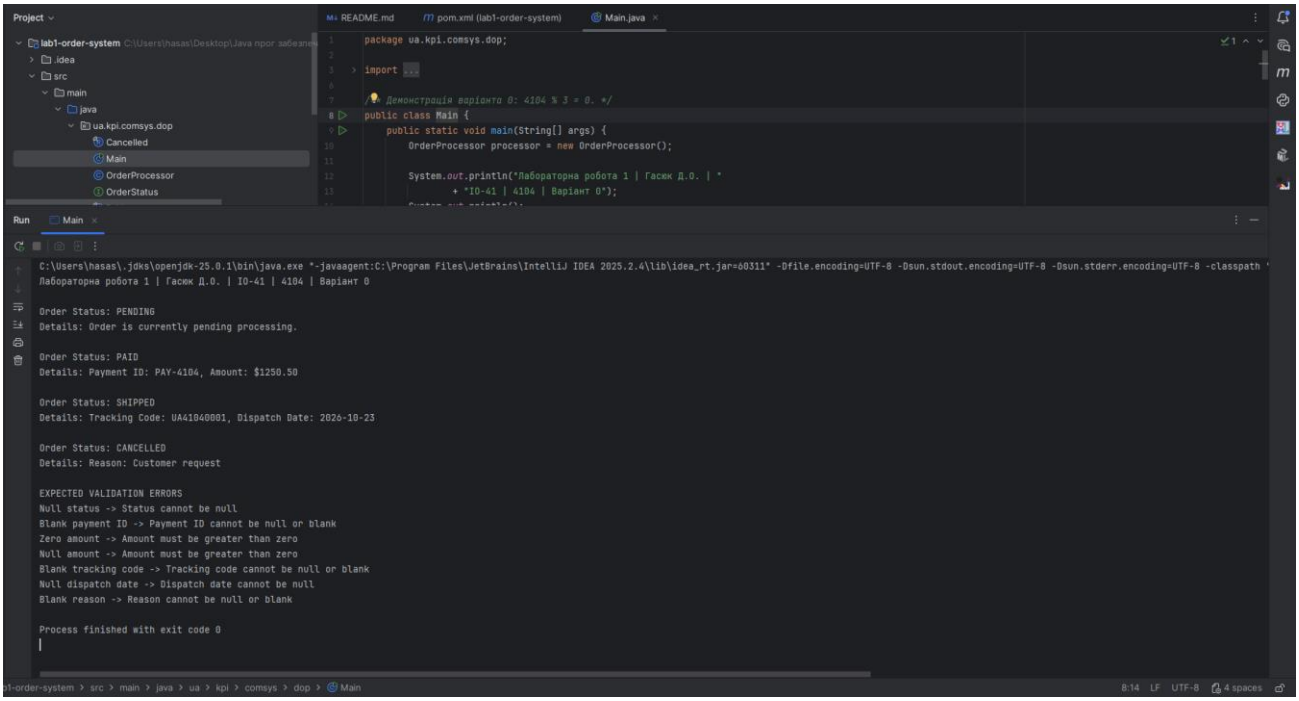
        // Сталі приклади дають однаковий результат під час кожного запуску.
        List<OrderStatus> statuses = List.of(
                new Pending(),
                new Paid("PAY-4104", new BigDecimal("1250.50")),
                new Shipped("UA41040001", LocalDate.of(2026, 10, 23)),
                new Cancelled("Customer request")
        );

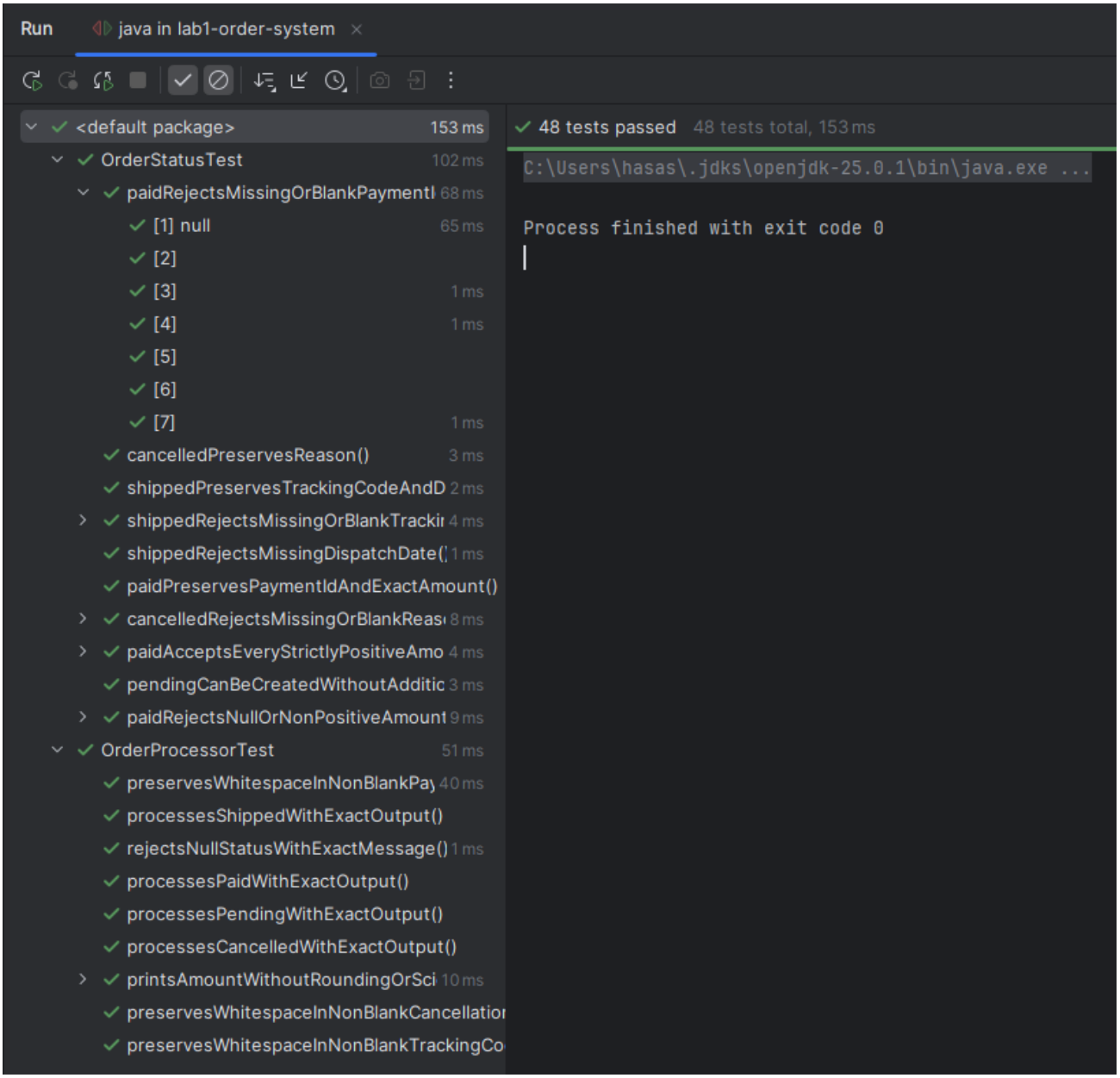
        for (OrderStatus status : statuses) {
            System.out.print(processor.process(status));
            System.out.println();
        }

        System.out.println("EXPECTED VALIDATION ERRORS");
        expectValidationError("Null status", () -> processor.process(null));
        expectValidationError("Blank payment ID",
                () -> new Paid("  ", BigDecimal.ONE));
        expectValidationError("Zero amount",
                () -> new Paid("PAY-4104", BigDecimal.ZERO));
        expectValidationError("Null amount", () -> new Paid("PAY-4104", null));
        expectValidationError("Blank tracking code",
                () -> new Shipped("\t", LocalDate.of(2026, 10, 23)));
        expectValidationError("Null dispatch date",
                () -> new Shipped("UA41040001", null));
        expectValidationError("Blank reason", () -> new Cancelled(" "));
    }

    // Очікувані винятки не переривають демонстрацію наступних прикладів.
    private static void expectValidationError(String example, Runnable action) {
        try {
            action.run();
        } catch (IllegalArgumentException exception) {
            System.out.println(example + " -> " + exception.getMessage());
            return;
        }
        throw new IllegalStateException(
                "Expected IllegalArgumentException: " + example);
    }
}
```

Скріншоти роботи програми:




Run
java in lab1-order-system
<default package> 153 ms
OrderStatusTest 102 ms
paidRejectsMissingOrBlankPaymentI 68 ms
[1] null 65 ms
[2]
[3] 1 ms
[4] 1 ms
[5]
[6]
[7] 1 ms
cancelledPreservesReason() 3 ms
shippedPreservesTrackingCodeAndD 2 ms
shippedRejectsMissingOrBlankTrackI 4 ms
shippedRejectsMissingDispatchDate(1 ms
paidPreservesPaymentIdAndExactAmount()
cancelledRejectsMissingOrBlankReas 8 ms
paidAcceptsEveryStrictlyPositiveAmo 4 ms
pendingCanBeCreatedWithoutAdditic 3 ms
paidRejectsNullOrNonPositiveAmount 9 ms
OrderProcessorTest 51 ms
preservesWhitespaceInNonBlankPa 40 ms
processesShippedWithExactOutput()
rejectsNullStatusWithExactMessage() 1 ms
processesPaidWithExactOutput()
processesPendingWithExactOutput()
processesCancelledWithExactOutput()
printsAmountWithoutRoundingOrSci 10 ms
preservesWhitespaceInNonBlankCancellatio
preservesWhitespaceInNonBlankTrackingCo
48 tests passed 48 tests total, 153 ms
C:\Users\hasas\.jdks\openjdk-25.0.1\bin\java.exe ...
Process finished with exit code 0

Висновки

У роботі реалізовано систему управління замовленнями для варіанта 0. Закрита ієрархія OrderStatus містить чотири публічні записи, які зберігають незмінні дані та перевіряють їх коректність у компактних конструкторах.

Обробник OrderProcessor відокремлює алгоритм форматування від моделі. Pattern matching у switch дає змогу визначати стан і отримувати його компоненти, а text blocks забезпечують читабельні шаблони результатів. Відсутність default дозволяє компілятору контролювати повноту обробки дозволених реалізацій sealed interface.

Компіляція та запуск підтвердили працездатність програми. Усі 48 тестових випадків JUnit 5 пройшли успішно. Перевірено формати результатів, обробку null, перевірки конструкторів і точне представлення десяткових сум.

Використані джерела

Лабораторна робота 1. Технології розробки на платформі Java. КПІ ім. Ігоря Сікорського. Умови завдання.

Oracle. Java SE 21 Language Updates. Record Classes. https://docs.oracle.com/en/java/javase/21/language/records.html

Oracle. Java SE 21 Language Updates. Pattern Matching for switch Expressions and Statements. https://docs.oracle.com/en/java/javase/21/language/pattern-matching-switch.html

JUnit Team. JUnit 5 User Guide 5.11.4. https://docs.junit.org/5.11.4/user-guide/